

PARTH MAHESH SAMBHUS

APEXPLANET CYBERSECURITY INTERNSHIP

TASK 3 – SECURITY TESTING REPORT

Web Application Security Assessment

Internship: ApexPlanet Cybersecurity Internship

Task: Task 3 – Web Application Security

Application Tested: Damn Vulnerable Web Application (DVWA)

Operating System: Kali Linux

Web Server: Apache

Database: MariaDB

Security Testing Tool: Burp Suite

Assessment Type: Controlled Local Laboratory Environment

1. INTRODUCTION

This report documents the web application security testing performed as part of Task 3 of the ApexPlanet Cybersecurity Internship.

The objective of this task was to gain practical experience in identifying, exploiting, and mitigating common web application vulnerabilities in a controlled laboratory environment.

Damn Vulnerable Web Application (DVWA) was deployed locally on Kali Linux and used as the intentionally vulnerable target application.

The assessment covered SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Local File Inclusion (LFI), Remote File Inclusion (RFI), Burp Suite request interception and fuzzing, and HTTP security headers.

All exploitation activities described in this report were performed against the local laboratory environment for educational and internship purposes.

2. OBJECTIVES

The main objectives of Task 3 were:

- Identify common web application vulnerabilities.
- Demonstrate SQL Injection attacks.
- Demonstrate Stored and Reflected XSS.
- Implement SQL Injection prevention using Prepared Statements.
- Implement XSS protection using input validation, output encoding, and Content Security Policy.
- Demonstrate a CSRF attack.
- Implement token-based CSRF protection.
- Demonstrate Local File Inclusion.
- Demonstrate Remote File Inclusion using a harmless controlled payload.
- Use Burp Suite to intercept and modify HTTP requests.
- Perform fuzzing using Burp Suite Intruder.
- Analyze HTTP security headers.
- Configure security headers in Apache.
- Document security findings and mitigations.

3. LAB ENVIRONMENT

Component	Details
Operating System	Kali Linux
Web Application	Damn Vulnerable Web Application (DVWA)
Web Server	Apache HTTP Server
Database	MariaDB
Security Testing Tool	Burp Suite
Security Header Analysis	SecurityHeaders.com
DVWA Security Level	Low
Environment	Local / Controlled Laboratory

The testing environment was isolated from production systems and was used only for cybersecurity training and demonstration.

4. METHODOLOGY

The following methodology was followed during the security assessment:

1. Identify the vulnerable functionality in DVWA.
2. Submit controlled security-testing input.
3. Observe the application's response.
4. Confirm the vulnerability.
5. Document the observed behavior.
6. Implement an appropriate security control or mitigation.
7. Test the mitigation.
8. Capture screenshots as evidence.
9. Document the findings in the GitHub repository.

5. SQL INJECTION

5.1 Objective

The objective was to demonstrate SQL Injection in DVWA and understand how attacker-controlled input can manipulate a vulnerable SQL query.

5.2 Vulnerability Demonstration

The DVWA SQL Injection module was tested using a UNION-based SQL Injection payload.

The following payload was used:

```
1' UNION SELECT user,password FROM users#
```

The payload successfully altered the SQL query and caused the application to return information from the users table.

5.3 Extracted Information

The following usernames and password hashes were returned during the controlled demonstration:

Username	Password Hash
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b

pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

5.4 Impact

SQL Injection can allow an attacker to:

- Retrieve unauthorized database information.
- Access sensitive user records.
- Manipulate application queries.
- Potentially modify or delete database information depending on database privileges.

5.5 Mitigation – Prepared Statements

A separate PHP demonstration was created using a prepared statement and parameterized query.

The query used a placeholder:

```
$stmt = $conn->prepare(
    "SELECT first_name, last_name FROM users WHERE user_id = ?"
);

$stmt->bind_param("s", $id);
```

The user input was therefore treated as a parameter rather than executable SQL syntax.

The malicious UNION-based input did not extract additional database records when tested against the protected implementation.

Output encoding using htmlspecialchars() was also applied.

5.6 Result

SQL Injection was successfully demonstrated in the vulnerable DVWA environment, and prepared statements were successfully demonstrated as a mitigation.

Evidence: SQL Injection screenshots are available in the Task 3/SQL_Injection/ directory of the GitHub repository.

6. CROSS-SITE SCRIPTING (XSS)

6.1 Stored XSS

Objective

The objective was to demonstrate a Stored Cross-Site Scripting vulnerability in DVWA.

Payload

The following JavaScript payload was submitted:

```
<script>alert('XSS Test')</script>
```

The application stored the supplied content and executed the JavaScript when the affected content was displayed.

Impact

Stored XSS can allow attacker-controlled JavaScript to execute in the browsers of users who view the affected content.

Depending on the application, XSS can potentially be used for:

- Malicious page modification.
- Phishing.
- Unauthorized browser actions.
- Session-related attacks.

6.2 Reflected XSS

Objective

The objective was to demonstrate Reflected Cross-Site Scripting using attacker-controlled input supplied through a request parameter.

Payload

The following payload was used:

```
<script>alert('Reflected XSS')</script>
```

The application reflected the supplied input into the response without sufficient protection, causing the JavaScript to execute.

Impact

Reflected XSS can occur when untrusted input is returned directly in a web response without appropriate output encoding or validation.

6.3 XSS Mitigation

A separate protected PHP page was created to demonstrate XSS mitigation.

Input Validation and Output Encoding

User-controlled input was processed using:

```
htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
```

This causes HTML special characters to be encoded so that supplied HTML or JavaScript is displayed as text instead of being interpreted as executable markup.

Content Security Policy

The following Content Security Policy was configured:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

The protected page was tested with the malicious JavaScript payload.

The payload was displayed as text instead of being executed, and the JavaScript alert did not appear.

Result

Both Stored XSS and Reflected XSS were successfully demonstrated in DVWA.

Input encoding and Content Security Policy were then used to demonstrate XSS mitigation.

Evidence: XSS screenshots are available in the Task 3/XSS/ directory of the GitHub repository.

7. CROSS-SITE REQUEST FORGERY (CSRF)

7.1 Objective

The objective was to demonstrate a Cross-Site Request Forgery vulnerability and implement token-based protection.

7.2 CSRF Attack

The vulnerable DVWA CSRF password-change functionality was tested while an authenticated DVWA session was active.

A separate local HTML page was created containing a forged password-change request.

The request included:

```
<input type="hidden" name="password_new" value="CSRFtest123">
<input type="hidden" name="password_conf" value="CSRFtest123">
<input type="hidden" name="Change" value="Change">
```

The forged request was submitted while authenticated to DVWA.

The application accepted the request and changed the password, demonstrating the CSRF vulnerability.

The password was restored to the original test password after the demonstration.

7.3 Impact

CSRF can allow an attacker to cause an authenticated user to perform an unintended action.

Potential examples include:

- Changing passwords.
- Modifying account information.
- Changing application settings.
- Performing unauthorized transactions.

7.4 CSRF Token Protection

A separate protected PHP page was created using a server-side CSRF token.

A random token was generated using:

```
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

The token was included as a hidden form field.

The submitted token was validated using:

```
if (!hash_equals($_SESSION['csrf_token'], $token)) {  
    die("CSRF validation failed.");  
}
```

Legitimate Request

A legitimate request containing the correct token was accepted successfully.

Forged Request

A simulated forged request without the required CSRF token was submitted.

The server rejected the request and displayed:

CSRF validation failed.

Result

The vulnerable CSRF behavior was successfully demonstrated and token-based protection successfully rejected a forged request without a valid token.

Evidence: CSRF screenshots are available in the Task 3/CSRF/ directory of the GitHub repository.

8. FILE INCLUSION

8.1 Local File Inclusion (LFI)

Objective

The objective was to demonstrate Local File Inclusion in DVWA.

The File Inclusion functionality was manipulated using the page parameter.

The following local file was requested:

```
/etc/passwd
```

The application successfully displayed the contents of the local `/etc/passwd` file.

This demonstrated that user-controlled input could be used to reference local files.

Impact

LFI can allow attackers to:

- Read sensitive local files.
- Disclose system information.
- Identify system accounts.
- Access application configuration files.
- Potentially combine the vulnerability with other attacks.

8.2 Remote File Inclusion (RFI)

Objective

The objective was to demonstrate Remote File Inclusion using a harmless controlled payload.

A PHP test file was created:

```
<?php  
echo "RFI Test Successful";  
?>
```

A local Python HTTP server was used to host the file:

```
cd /var/www/html
python3 -m http.server 8000
```

The DVWA File Inclusion module was then supplied with the URL of the controlled test file.

The application successfully executed the controlled PHP payload and displayed:

RFI Test Successful

Mitigation

Recommended protections include:

- Avoiding user-controlled file paths.
- Using allowlists for permitted files.
- Validating and canonicalizing file paths.
- Disabling remote file inclusion when it is not required.
- Maintaining secure PHP configuration.

After testing, the temporary PHP file and Python server were removed/stopped and the PHP configuration was restored.

Result

Both LFI and RFI were successfully demonstrated in the controlled laboratory environment.

Evidence: File Inclusion screenshots are available in the Task 3/File_Inclusion/ directory of the GitHub repository.

9. BURP SUITE ADVANCED

9.1 Objective

Burp Suite was used to inspect, intercept, modify, and fuzz HTTP requests against the local DVWA application.

9.2 HTTP Request Interception

Burp Suite's built-in browser was used to access the DVWA login page.

The login traffic was intercepted through:

Proxy → Intercept

The HTTP request was inspected before being forwarded to the application.

This demonstrated how Burp Suite can be used to observe communication between a browser and web application.

9.3 HTTP Request Modification

The intercepted login request was further analyzed.

The username parameter was modified from:

admin

to:

testuser

The modified request was sent to the server and the resulting response was observed.

This demonstrated how an authorized security tester can modify HTTP parameters during application testing.

9.4 Intruder Fuzzing

Burp Suite Intruder was used to test multiple username inputs against the DVWA login endpoint.

The username parameter was selected as the fuzzing position and multiple candidate values were supplied.

The resulting responses were observed and compared.

Purpose

Intruder can be used during authorized security testing to:

- Automate repetitive request testing.
- Test input handling.
- Identify unexpected application behavior.
- Compare responses to different inputs.

Result

Burp Suite was successfully used for:

- HTTP request interception.
- Request inspection.
- Request modification.
- Intruder fuzzing.

Evidence: Burp Suite screenshots are available in the Task 3/Burp_Suite/ directory of the GitHub repository.

10. WEB SECURITY HEADERS

10.1 Security Header Analysis

A public test website was analyzed using SecurityHeaders.com to understand how HTTP security headers are assessed.

The analysis provided an overview of security-related HTTP headers and their configuration.

10.2 Apache Security Header Configuration

Apache's mod_headers module was enabled using:

```
sudo a2enmod headers
```

A dedicated Apache configuration file was created:

```
/etc/apache2/conf-available/security-headers.conf
```

The following headers were configured:

```
<IfModule mod_headers.c>  
  Header always set X-Content-Type-Options "nosniff"  
  Header always set X-Frame-Options "SAMEORIGIN"  
  Header always set Referrer-Policy "strict-origin-when-cross-origin"  
  Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"</IfModule>
```

The configuration was enabled with:

```
sudo a2enconf security-headers
```

The Apache configuration was tested using:

```
sudo apache2ctl configtest
```

The result was:

Syntax OK

Apache was then restarted successfully.

10.3 Verification

The configured headers were verified using:

```
curl -I http://127.0.0.1/dvwa/
```

The response contained:

```
X-Content-Type-Options: nosniff  
X-Frame-Options: SAMEORIGIN  
Referrer-Policy: strict-origin-when-cross-origin  
Permissions-Policy: geolocation=(), microphone=(), camera=()
```

This confirmed that the configured HTTP security headers were being returned by the Apache server.

10.4 Purpose of the Security Headers

X-Content-Type-Options

The nosniff value helps prevent browsers from MIME-sniffing responses and interpreting content as a different type than declared.

X-Frame-Options

The SAMEORIGIN value restricts framing of the application to the same origin and can help reduce clickjacking risks.

Referrer-Policy

The strict-origin-when-cross-origin policy controls how much referrer information is sent with requests, particularly when requests cross origins.

Permissions-Policy

The configured policy restricts access to selected browser features such as geolocation, microphone, and camera.

Result

Security headers were successfully configured in Apache and verified through an HTTP response from the local DVWA application.

Evidence: The Apache security-header verification screenshot is available in the Task 3/Security_Headers/ directory of the GitHub repository.

11. FINDINGS SUMMARY

Security Area	Test Performed	Mitigation / Defensive Measure
SQL Injection	UNION-based SQL Injection	Prepared Statements
Stored XSS	Stored JavaScript execution	Output Encoding + CSP

Reflected XSS	Reflected JavaScript execution	Output Encoding + CSP
CSRF	Forged password-change request	CSRF Token
LFI	/etc/passwd disclosure	Input Validation + File Allowlisting
RFI	Controlled remote PHP inclusion	Disable unnecessary remote inclusion
Burp Suite	HTTP interception and modification	Request Analysis
Intruder	Username fuzzing	Input Testing and Monitoring
Security Headers	HTTP header analysis	Apache Security Headers

12. KEY LEARNING OUTCOMES

Through this task, the following practical skills were developed:

- Understanding common web application vulnerabilities.
- Identifying SQL Injection vulnerabilities.
- Using UNION-based SQL Injection in a controlled environment.
- Understanding the importance of prepared statements.
- Identifying Stored and Reflected XSS.
- Applying output encoding and Content Security Policy.
- Understanding CSRF attacks and CSRF tokens.
- Identifying Local File Inclusion vulnerabilities.
- Understanding Remote File Inclusion.
- Using Burp Suite Proxy for HTTP interception.
- Modifying HTTP requests during authorized testing.
- Using Burp Suite Intruder for fuzzing.
- Understanding HTTP security headers.
- Configuring Apache security headers.
- Verifying server configuration using command-line tools.
- Documenting security testing activities and mitigation techniques.

13. EVIDENCE AND DOCUMENTATION

The evidence collected during the assessment has been organized in the GitHub repository under the Task 3 directory.

The following folders contain the corresponding documentation and screenshots:

Task 3/

- |
- |— SQL_Injection/
- |— XSS/
- |— CSRF/
- |— File_Inclusion/
- |— Burp_Suite/
- |— Security-Headers/

Each folder contains a README file explaining the relevant security testing activity and the associated screenshots.

14. CONCLUSION

Task 3 provided hands-on experience with web application security testing in a controlled DVWA laboratory environment.

The assessment demonstrated multiple common web application vulnerabilities, including SQL Injection, Stored and Reflected XSS, CSRF, Local File Inclusion, and Remote File Inclusion.

Burp Suite was used to intercept, inspect, modify, and fuzz HTTP requests. Apache security headers were also configured and verified to demonstrate server-side defensive controls.

For the vulnerabilities demonstrated, appropriate mitigation techniques were implemented where required, including prepared statements, output encoding, Content Security Policy, CSRF tokens, secure file handling practices, and HTTP security headers.

The completed work demonstrates practical experience in both offensive web application security testing and defensive security implementation.

All activities documented in this report were performed within the controlled local laboratory environment for educational and internship purposes.

15. DECLARATION

I confirm that the security testing activities documented in this report were performed in a controlled laboratory environment using an intentionally vulnerable application for educational and internship purposes.

The techniques demonstrated were used to understand web application vulnerabilities, their potential impact, and appropriate defensive measures.